

The F1R3Skein Virtual Instrument

Specification, revision 2

F1R3FLY.io

4 September 2026

Abstract

This document specifies the F1R3Skein virtual instrument: the gestural surface through which a player draws melody out of the digit streams of mathematical constants, and out of which the platform’s two primary artifacts—tunes and performances—are produced. It supersedes the behaviour described in the existing `LeapSpigotVision` README, which is inaccurate in several respects, and it is written to be checked rather than admired: conformance language is normative, every threshold is named and externalised, and the wire protocol is given in full.

Revision 2 makes four substantive changes to what is currently implemented. The transport moves off the Unix domain socket, which cannot cross a device boundary. Audio moves onto the headset. The gesture vocabulary gains explicit mutual exclusion and a calibration profile. And the semantics of the snip gesture are corrected: as implemented, **a snipped tune is not the tune the player heard**, and Section 6 sets out why, together with the resolution proposed for adoption.

Section 14 lists the seven points on which agreement is sought before implementation begins.

Contents

1	Scope and conformance	2
1.1	Scope	2
1.2	Conformance language	2
1.3	Terminology	2
2	Deployment configurations	3
3	The generative core	3
3.1	Streams	3
3.2	The skein and its two cursors	3
3.3	Note mapping	4
4	Instrument state	4
4.1	State vector	4
4.2	Playback state machine	4
5	Gesture vocabulary	5
5.1	The six gestures	5
5.2	Mutual exclusion	5
5.3	Detection parameters	5
5.4	Finger curl must be scale-invariant	6

6 Snip semantics	6
6.1 The defect	6
6.2 Resolution	6
6.3 Consequences of (b)	7
6.4 Naming	7
7 Performance recording	7
8 Wire protocol, version 2	8
8.1 Framing and encoding	8
8.2 Client to engine	8
8.3 Engine to client	8
8.4 Rate discipline	9
9 Visual model	9
10 Audio model	9
11 Non-functional requirements	10
12 Instrumentation	10
13 Conformance checklist	10
14 Decisions sought	11

1 Scope and conformance

1.1 Scope

In scope: the generative core, the instrument’s state model, the gesture vocabulary and its detection criteria, the client/engine wire protocol, the visual and audio models, the recording of performances, and the non-functional budgets that make the instrument playable.

Out of scope, and specified elsewhere: identity and sign-on, the galleries, the engagement ledger, breeding and selection, and the on-chain representation of tunes. This document defines what the instrument *produces*; the development plan defines what becomes of it.

1.2 Conformance language

MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are used in the sense of RFC 2119. A requirement labelled MUST is a conformance condition for the September test build.

1.3 Terminology

<i>Engine</i>	The Rust process (or on-device library) holding the authoritative instrument state.
<i>Client</i>	The visionOS application: gesture recognition, rendering, and audio.
<i>Stream</i>	One spigot: a constant, an output base, and a cursor position.
<i>Skein</i>	The pair of streams, with a designation of which governs pitch and which duration.
<i>Ribbon</i>	The visual presentation of one stream: a receding row of coloured digit patches.
<i>Tune</i>	A snipped, named artifact. The unit of storage, breeding and gallery display.

Performance

The recorded trace of a session at the instrument.

2 Deployment configurations

Two configurations are specified. Both **MUST** be supported by the same engine code and the same protocol; they differ only in transport and in where the engine runs.

S1 — Tethered.

Engine on a Mac, client on the headset, connected over TCP on the local network with Bonjour discovery. This is the September test configuration. It is the fast path and it is explicitly a stopgap.

S2 — On-device.

Engine compiled for `arm64-apple-visionos` as a static library and called through a C foreign-function interface. No transport, no network, no discovery. This is the target configuration.

The protocol of Section 8 is defined over an abstract bidirectional message channel. In S1 that channel is a TCP stream; in S2 it is a pair of FFI ring buffers. **No other part of this specification distinguishes the two**, and the client **MUST NOT** contain transport-specific logic outside its channel implementation.

The existing Unix-domain-socket transport **MUST NOT** be used for device testing. A Unix socket is addressed by a path in one host’s filesystem; the headset is a different host. It **MAY** be retained for engine development on the Mac.

3 The generative core

3.1 Streams

The engine **MUST** provide the five constants already implemented— π , e , φ , $\ln 2$, and Liouville’s constant—each emitting digits in a configurable output base b with $2 \leq b \leq 36$. Digit emission **MUST** be exact and unbounded: no fixed-precision truncation, and no upper limit on cursor position other than time and memory.

Definition 1 (Stream configuration). A stream configuration is a pair (c, b) of a constant and a base. Together with a cursor position $p \in \mathbb{N}$ it determines the digit $d(c, b, p)$ uniquely.

The π implementation currently uses Gosper’s unbounded linear-fractional-transformation spigot with arbitrary-precision integers, and is materially slower than the other four. Since deep cursor positions are exactly what a long session produces, the engine **SHOULD** adopt a Bailey–Borwein–Plouffe extraction for π in bases where it applies, and **MUST** in any case memoise emitted digits so that re-realisation of an already-visited range is $O(1)$ per digit.

3.2 The skein and its two cursors

The skein holds two configurations and two *independent* cursor positions, p_L and p_R . Independence is the whole point of the two-handed design: the left hand advances one stream, the right hand the other, and the player’s control over the music is precisely their control over the relative rate of the two.

Consequence, and it is load-bearing: at any moment p_L and p_R are in general *different*, and the difference is the player’s doing. Any operation that assumes a single shared position is wrong. Section 6 is about the one place this assumption was made.

3.3 Note mapping

By convention one stream governs pitch and the other duration; *twist* exchanges the roles. The engine MUST produce, for the k -th zipped pair, a note

$$(\text{pitch} = \Pi(d_{\text{pitch}}), \quad \text{duration} = \Delta(d_{\text{duration}}))$$

where Π is a pitch map and Δ a duration map. Both MUST be parameters of the session, not constants of the build. Pitch maps MUST include at minimum major, minor, pentatonic major, pentatonic minor, Dorian, whole tone and chromatic, each over a configurable root. Duration maps MUST include musical, linear, exponential and fixed.

The base of a stream SHOULD be chosen to match the cardinality of the map it drives: a 22-element pitch vocabulary is naturally driven by a base-22 stream, a 5-element rhythmic vocabulary by a base-5 stream. Mismatches are permitted and produce a modular fold, which is musically legitimate but should be a choice rather than an accident.

Decision required. D-1. Whether Π and Δ are fixed for the duration of a performance, or are themselves gesture-modulable mid-performance (“change scale with a gesture”). The latter is musically far richer and is a natural extension of the vocabulary, but it adds state that the tune representation must then carry. See Section 14.

4 Instrument state

4.1 State vector

The engine’s authoritative state MUST be exactly:

$$\Sigma = \langle (c_L, b_L, p_L), (c_R, b_R, p_R), \rho, \Pi, \Delta, \tau, \iota, T \rangle$$

with $\rho \in \{\text{Stopped}, \text{Playing}\}$ the playback state, τ the tempo, ι the instrument (General MIDI program), and T the tune tray. The client MUST NOT hold authoritative state; it renders Σ and emits gestures.

4.2 Playback state machine

State	Event	Next	Effect
Stopped	Clap	Playing	begin realisation; ribbons stitch
Stopped	Unclap	Stopped	ignored
Playing	Unclap	Stopped	halt realisation; ribbons separate
Playing	Clap	Playing	ignored
any	Pull, Twist, Scissors	unchanged	see §5

Pulling MUST be permitted while playing; that is how a player steers a phrase as it sounds.

5 Gesture vocabulary

5.1 The six gestures

Gesture	Physical form	Effect
Pull left	Left hand drawn toward the player	Advance the left stream; step count proportional to speed
Pull right	Right hand drawn toward the player	Advance the right stream, likewise
Twist	One hand held clearly above the other, hands apart	Exchange the two streams, cursors included
Clap	Hands brought together	Enter Playing
Unclap	Hands drawn apart from a clap	Enter Stopped
Scissors	Index and middle extended and spread, ring and little curled	Snip the current window into a named tune

5.2 Mutual exclusion

The current implementation evaluates clap and twist independently, so hands brought together with one slightly higher—an entirely natural motion—fires both. The following rules **MUST** hold:

1. Twist **MUST** require hand separation greater than the unclap distance. Twist and clap are therefore never simultaneously satisfiable.
2. Scissors **MUST** be suppressed while the hands are within clap distance of one another.
3. Pull **MUST** be suppressed for a hand that is participating in a clap transition in the same frame.
4. Every gesture **MUST** be edge-triggered with a cooldown, and **MUST NOT** re-fire while its pose is continuously held.

5.3 Detection parameters

Every threshold **MUST** live in an external calibration profile, loaded at launch and reloadable without a rebuild. Hardcoded constants in `GestureRecognizer.swift` **MUST NOT** remain.

Parameter	Governs	Provisional
<code>clap.distance</code>	hands-together threshold	0.10 m
<code>unclap.distance</code>	hands-apart threshold	0.22 m
<code>pull.velocity_min</code>	minimum speed toward player	0.25 m/s
<code>pull.step_divisor</code>	speed to step count	0.15
<code>pull.cooldown</code>	minimum interval	0.08 s
<code>twist.height_delta</code>	vertical offset	0.06 m
<code>twist.hold_frames</code>	frames before firing	8
<code>scissors.spread</code>	index/middle angle	0.40 rad
<code>scissors.extended</code>	extension floor, index/middle	<i>to be derived</i>
<code>scissors.curled</code>	curl ceiling, ring/little	<i>to be derived</i>
<code>scissors.hold_frames</code>	frames before firing	5
<code>scissors.cooldown</code>	minimum interval	0.60 s

5.4 Finger curl must be scale-invariant

The implemented curl metric is $\|\text{tip} - \text{metacarpal}\|/0.08$, clamped to unity, with “curled” asserted below 0.30. On an adult hand a fully curled ring finger measures roughly 0.06–0.08 m tip-to-metacarpal, giving 0.75–1.0. The threshold is unreachable, and scissors consequently never fires from hand tracking. The metric is also not scale-invariant: the fixed 0.08 m divisor makes detection depend on hand size.

Curl MUST instead be measured either as the interior angle at the proximal interphalangeal joint, or as tip-to-wrist distance normalised by that finger’s own summed segment lengths. Both are scale-invariant. Thresholds MUST be derived from recorded traces (§12), not chosen by inspection.

6 Snip semantics

6.1 The defect

This is the most important paragraph in the document.

The engine derives a snip as follows. `do_snip` computes $\text{from} = p_L - w$ and $\text{to} = p_L$, where w is the ribbon width, and then calls `DualStream::snip(name, from, to)`. That function creates fresh spigots from each side’s configuration, advances *both* to *from*, and collects *to*–*from* pairs.

Both sides are advanced to a position computed *from the left cursor alone*. But p_L and p_R are independent by design—the player has been moving them at different rates all session. So the stored snippet is the sequence

$$\left(d(c_L, b_L, k), d(c_R, b_R, k) \right)_{k \in [p_L - w, p_L]}$$

whereas what the player heard and saw was

$$\left(d(c_L, b_L, i), d(c_R, b_R, j) \right)$$

with i and j advancing independently. These coincide only if the two cursors happen to have moved in lockstep, which is precisely the case the instrument exists to avoid.

A snipped tune is, in general, not the tune the player heard. It is a different tune, derived from the same two constants.

There is a third representation in play, which compounds the problem. The tray receives the digits actually *displayed*—`left_ribbon.patches` zipped with `right_ribbon.patches`—while the snippet map receives the re-derived pairs above. So “the snipped tune” currently denotes three different things depending on where one looks: what was heard, what is in the tray, and what is in the snippet map.

6.2 Resolution

Three resolutions are available.

(a) Zip-index snip. Maintain a log of realised pairs and snip an interval of that log. Exactly faithful to what was heard. Costs a growing log, and—decisively—destroys the address property: a tune is no longer reconstructible from a handful of numbers, because the pairing is arbitrary.

(b) Independent intervals. *Recommended.* A snip records both cursor positions and a common length:

$$\text{Snip}(k, i_L, i_R, n) \text{ denoting } (d(c_L, b_L, i_L + t), d(c_R, b_R, i_R + t))_{t \in [0, n]}.$$

Since both ribbons are fed by the same advance operations that drive realisation, the window $[p_L - n, p_L) \times [p_R - n, p_R)$ contains exactly the pairs displayed and sounded. Faithful, and the address property survives: four numbers and two configurations.

(c) Lockstep. Forbid independent advancement. Correct by construction and rejected: it deletes the two-handed idea that the instrument is for.

6.3 Consequences of (b)

Adopting (b) requires three changes beyond the engine.

The tray, the snippet map and the realised audio **MUST** all be derived from the same $\text{Snip}(k, i_L, i_R, n)$, eliminating the three-way divergence.

The Theory of skeins requires amendment. **Skein.module** currently declares

```
Snip . k:Skein, i:Nat, j:Nat |- "snip" "(" k ", " i ", " j ")" : Tune;
```

which inherits the single-interval assumption. It becomes

```
Snip . k:Skein, iL:Nat, iR:Nat, n:Nat
  |- "snip" "(" k ", " iL ", " iR ", " n ")" : Tune;
```

with the empty-snip equation restated as $(\text{Snip } k \ i_L \ i_R \ \text{NZero}) == (\text{Rest})$.

Twist then interacts correctly and for free:

Proposition 1. *Under (b), snipping commutes with twisting: normalising $\text{Twist}(\text{Wv } l \ r)$ to $\text{Wv } r \ l$ also reinterprets i_L as indexing r , which is exactly what the engine does, since *twist* swaps whole spigots—configuration and cursor together. No additional equation is needed.*

Decision required. **D-2.** Adopt resolution (b). Confirm, and confirm the amendment to **Skein.module**.

6.4 Naming

A scissors gesture **MUST** suspend nothing: realisation continues, cursors keep their positions, and the player is prompted for a name asynchronously. If no name is supplied within a timeout the engine **MUST** assign a provisional one rather than discard the snip. Losing a capture because a text field was dismissed is the worst available outcome.

7 Performance recording

Definition 2 (Performance trace). An ordered sequence of timestamped records: an opening declaration of both stream configurations, the pitch and duration maps, tempo and instrument; followed by every gesture event with its parameters; followed by a closing timestamp.

The engine **MUST** record a trace for every session, without the player asking. Because the streams are deterministic and the state machine is total, the trace **MUST** replay the session exactly; no audio is stored. Snips occurring during a session **MUST** appear in the trace, so that a performance resolves to the tunes harvested from it and each tune resolves back to its performance.

Trace records **MUST** carry monotonic timestamps at millisecond resolution or better, and **SHOULD** additionally carry the raw hand-tracking metrics that produced each gesture, for the purpose of §12.

8 Wire protocol, version 2

8.1 Framing and encoding

Messages **MUST** be newline-delimited JSON, one object per line, each carrying a **type** field. Both sides **MUST** serialise via **Codable** and **serde** respectively; neither side **MAY** construct JSON by string interpolation. The current Swift client builds gesture JSON by interpolation and escapes only the double quote, so a tune name containing a backslash produces malformed JSON that the engine silently discards.

Every message **MUST** carry "v": 2. A client receiving an unrecognised version **MUST** report it rather than degrade.

8.2 Client to engine

Type	Fields
pull_left, pull_right	steps, velocity
twist, clap, unclap	—
scissors	name
configure	left, right, pitch_map, duration_map, tempo, instrument
quit	—

configure is new and **MUST** be supported: the instrument is currently configured only by command-line flags on the engine, which is unusable from a headset.

8.3 Engine to client

Type	Fields
digits	left, right, left_pos, right_pos
note	pitch, duration, velocity, left_pos, right_pos, patch_index
snip_ack	name, count, i_left, i_right
state	playing, left_label, right_label, tempo, instrument
status	text
error	code, text

Four changes from version 1, each fixing a defect:

1. **state** replaces the practice of inferring playback state by substring-matching the status text. The client currently clears its playing flag on `statusText.contains("stopped")`, which works only until someone rewords a message. Playback state **MUST** be a boolean field.
2. **note** gains **patch_index**. The engine already computes the highlighted patch and does not transmit it, which is why the playing-note cursor is unimplemented on the client.
3. **snip_ack** gains the two cursor positions, per §6.
4. **status** becomes advisory and human-readable only. Nothing in the client **MAY** parse it.

8.4 Rate discipline

The engine currently emits a **status** message on every frame of its 60 Hz loop, beneath a comment reading “Push status if changed” which describes a check the code does not perform. Each message invalidates the entire client panel.

- **status** and **state** MUST be emitted only on change.
- **digits** MUST be emitted only when a cursor moves, and MUST be coalesced to at most 30 Hz.
- **note** MUST be emitted per note and MUST NOT be coalesced.
- Aggregate steady-state traffic SHOULD remain below 200 messages per second.

9 Visual model

Four elements are specified. All four exist in the current source; three are never called.

Ribbons. Two receding rows of digit patches, one per stream, coloured by digit value on a hue wheel and faded with depth. Patch entities MUST be pooled and reused. Text meshes and materials MUST be cached per digit value and per depth bucket, and MUST NOT be regenerated on every update—the current `PatchEntity.configure` calls `MeshResource.generateText` and constructs a fresh physically-based material for every patch on every frame, which is the dominant cost in the render path.

Stitching. On entering **Playing**, threads MUST animate between the two ribbons, and MUST unstitch on leaving. This is the player’s only confirmation that a clap registered, and it is currently not wired.

Scissor highlight. A snip MUST highlight the captured window in gold and animate its deposit into the tray. Also implemented, also not wired.

Hand ghosts. Wireframe hand skeletons MUST mirror the tracked joints. The gesture recogniser receives every hand anchor and discards it; the ghost entity is added to the scene and never updated. Wiring these is a matter of forwarding anchors the recogniser already holds.

The tray MUST display the digits actually snipped. It currently displays the live ribbon contents, which after any subsequent pull are not the same thing.

10 Audio model

Audio MUST be synthesised on the headset. In the current architecture the engine drives CoreMIDI on the Mac, so the player pulls a ribbon in front of their face and hears the result from a laptop across the room, after a network hop and a MIDI buffer. For an instrument this is disqualifying, and it makes the latency figure—the single most important measurement of the September sessions—unrepresentative of any shipping configuration.

- The client MUST render **note** messages through an on-device sampler.
- Audio SHOULD be spatialised so that each ribbon sounds from its own position in the immersive scene. The two streams are distinct objects in front of the player and should be audibly distinct.
- Mac-side MIDI output MAY be retained as an option for recording to a digital audio workstation, and MUST NOT be the default.

11 Non-functional requirements

Property	Target	Maximum
Gesture to audible note	30 ms	50 ms
Gesture to visual response	one frame	two frames
Rendered frame rate	90 fps sustained	—
Digit emission, memoised range	—	1 ms per 1000 digits
Engine to client message rate	$< 100 \text{ s}^{-1}$	200 s^{-1}
Session length without degradation	45 min	—

Failure modes **MUST** be visible and recoverable. Loss of the channel **MUST** surface in the panel and **MUST** retry with backoff, preserving engine state across reconnection. Denial of hand-tracking authorization **MUST** produce an explanatory message and fall back to the panel controls, rather than a silent console line as at present. The tune tray **MUST NOT** silently discard entries; it currently caps at twenty and drops the oldest, which over a long session will quietly delete the early captures.

12 Instrumentation

The gesture thresholds are guesses, one of them is provably unreachable, and there exists no data against which to tune them. The instrument **MUST** therefore be able to record its own inputs.

- A debug overlay **MUST** display, live, the computed metrics for each hand: separation, palm velocity, height differential, per-finger extension and curl, and which gesture predicates are currently satisfied.
- Raw joint transforms **MUST** be recordable to disk at tracking rate, with a manual label marker so a tester can annotate “that was a scissors” as it happens.
- Thresholds **MUST** be tunable offline against recorded traces and reloadable without a rebuild.

The first half-day of headset time **SHOULD** be spent capturing a labelled corpus rather than playing. Recorded traces are the durable output of the test window: they are the same artifact as a performance trace, so this work is not a detour from the product but a first delivery of it.

13 Conformance checklist

A build is ready for device testing when all of the following hold.

1. Clean install on device succeeds; entitlements and provisioning verified.
2. Hand-tracking authorization is requested, granted, and denial is handled visibly.
3. The engine is discovered automatically; the panel indicates connection within five seconds.
4. All six gestures fire, each confirmed in the debug overlay, before any music is attempted.
5. Hand ghosts are visible and track.
6. Clap produces both stitching and audio from the headset.
7. Scissors produces a name prompt; the named tune appears in the tray with the correct pair count and the correct digits.

8. A snipped tune, re-realised from its recorded (k, i_L, i_R, n) , is identical to what was heard at the moment of the snip.
9. 90 fps sustained with sixty patches per ribbon.
10. Tune tray and performance trace both survive an application restart.
11. Gesture traces are being written to disk.

Item 8 is the acceptance test for Section 6 and cannot pass on the current build.

14 Decisions sought

1. **Snip semantics.** Adopt resolution (b), independent intervals $\text{Snip}(k, i_L, i_R, n)$, and amend `Skein.module` accordingly?
2. **Configuration ordering.** S1 tethered for September with S2 on-device to follow, or go straight to S2 and accept the risk to the test window? S1 is roughly three days; S2 is roughly a week and retires the whole transport question.
3. **Modulable maps.** Should pitch and duration maps be gesture-modulable mid-performance (D-1)? If yes, the tune representation must carry them, which touches the Theory.
4. **Ribbon width.** The snip window is currently the ribbon width, so the visible ribbon *is* the capture buffer. Is that the intended relationship, or should the capture window be independently adjustable—and if so, by what gesture?
5. **Twist during playback.** Permitted, exchanging roles mid-phrase? The state machine above permits it. It is musically interesting and makes the resulting term slightly harder to read.
6. **Performance recording consent.** Recording every session by default is proposed here. Confirm, and confirm whether traces leave the device before the player chooses to publish.
7. **Naming timeout.** What provisional name, and after how long, when a player does not name a snip?